2026-9-18 Linux网络--TCP Socket编程与工程实践

- 1. TCP Socket 编程与工程实践
- 2. Linux 进程、作业与会话
  - 2.1 进程组与作业
    - 2.1.1 PID、PGID 与 SID
    - 2.1.2 作业号与前后台
    - 2.1.3 TTY、TPGID 与进程状态
  - 2.2 登录会话与终端
    - 2.2.1 SSH 登录链路
    - 2.2.2 注销为什么影响服务
- 3. 守护进程与长期运行
  - 3.1 setsid
  - 3.2 标准流、工作目录与日志
  - 3.3 部署目录
  - 3.4 服务重启与地址复用
- 4. Socket API
  - 4.1 文件描述符与地址
    - 4.1.1 socket
    - 4.1.2 sockaddr_in
  - 4.2 服务端
    - 4.2.1 socket、bind、listen
    - 4.2.2 accept
  - 4.3 客户端
    - 4.3.1 connect
    - 4.3.2 数据收发
- 5. 服务端并发模型
  - 5.1 单连接版本
  - 5.2 多进程版本
  - 5.3 多线程版本
  - 5.4 线程池版本
  - 5.5 长连接处理
- 6. Windows TCP 客户端
  - 6.1 Winsock 初始化
  - 6.2 Linux 与 Windows 差异
- 7. 断线检测与重连
  - 7.1 连接状态机

    - 7.2 重连流程
    - 7.3 示例的改进
    - 7.4 应用层心跳
- 8. 内核中的 socket
    - 8.1 文件描述符链路
    - 8.2 tcp_sock 保存的状态
    - 8.3 sk_buff
- 9. 工程实践检查表
    - 9.1 地址与连接
    - 9.2 数据收发
    - 9.3 资源管理
    - 9.4 断线与重连
- 10. 小结

1. TCP Socket 编程与工程实践

本篇从应用开发者视角讨论 TCP：

- Linux 服务进程如何脱离终端长期运行；
- `socket/bind/listen/connect/accept` 分别做什么；
- 如何从单连接服务器演进到进程、线程和线程池；
- Windows 客户端如何访问 Linux 服务；
- 如何处理短读短写、断线和重连；
- 文件描述符最终如何映射到内核 TCP 对象。

2. Linux 进程、作业与会话

2.1 进程组与作业

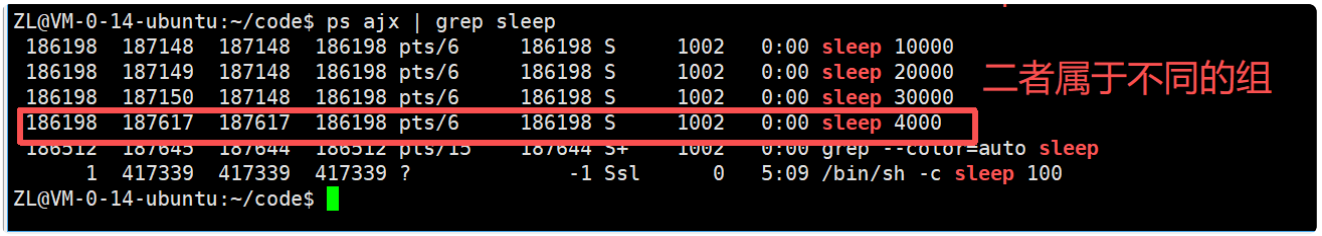
2.1.1 PID、PGID 与 SID

Linux 为每个进程分配 PID，父进程 PID 记录在 PPID 中。多个进程可以属于同一进程组，通过 PGID 标识；进程组属于会话，通过 SID 标识。

```
PPID    PID     PGID    SID     TTY     STAT    COMMAND
3964528 3964531 3964531 3964531 pts/0   Ss      -bash
```

```
 3964531 3965269 3964528 3964531 pts/0    S        sleep 10000
 3964531 3965268 3964528 3964531 pts/0    S        sleep 20000
```

进程组不等于组长进程。只要组内仍有成员，组长退出后进程组仍可能存在。

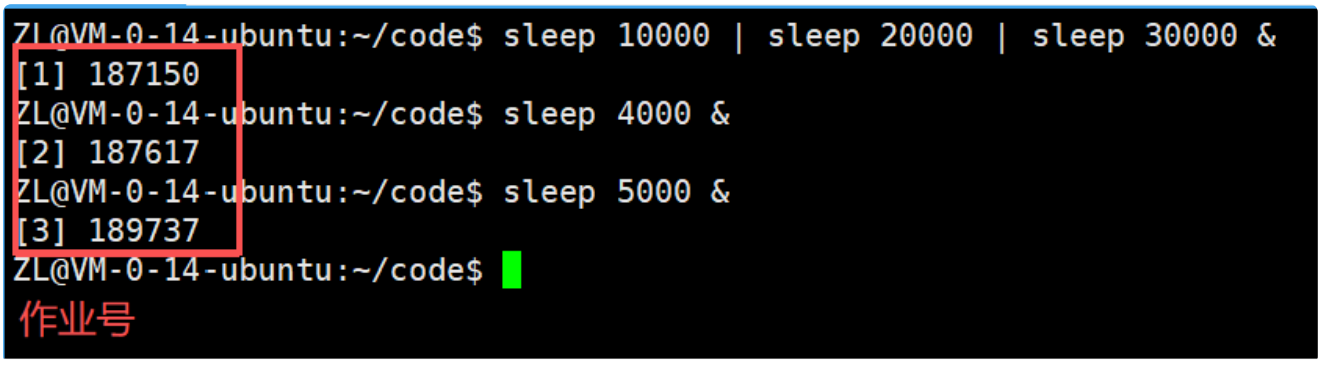


2.1.2 作业号与前后台

shell 用作业管理一组命令，作业通常由一个进程组承担。作业强调要完成什么任务，进程组强调任务如何被组织。

使用 `&` 将任务放入后台，使用 `jobs` 查看：

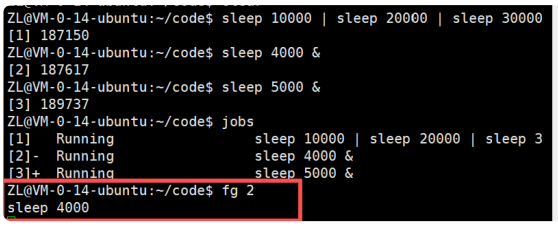
```
sleep 10000 &
sleep 20000 &
jobs
```

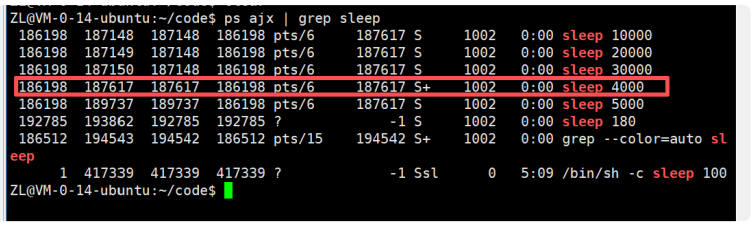



```
ZL@VM-0-14-ubuntu:~/code$ sleep 10000 | sleep 20000 | sleep 30000 &
[1] 187150
ZL@VM-0-14-ubuntu:~/code$ sleep 4000 &
[2] 187617
ZL@VM-0-14-ubuntu:~/code$ sleep 5000 &
[3] 189737
ZL@VM-0-14-ubuntu:~/code$ jobs
[1]   Running                 sleep 10000 | sleep 20000 | sleep 30000 &
[2]-  Running                 sleep 4000 &
[3]+  Running                 sleep 5000 &
ZL@VM-0-14-ubuntu:~/code$
```

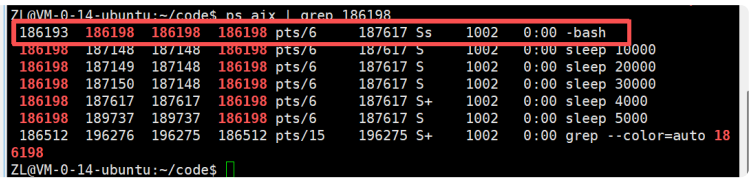
命令	作用
`jobs`	查看当前 shell 的作业
`fg %作业号`	把作业切到前台
`bg %作业号`	让暂停作业在后台继续
`Ctrl+C`	向前台进程组发送 `SIGINT`
`Ctrl+Z`	向前台进程组发送 `SIGTSTP` 并暂停

暂停后的作业转入后台，否则一个暂停的前台进程仍占用终端输入权，shell 无法继续读取命令。









2.1.3 TTY、TPGID 与进程状态

字段	含义
TTY	控制终端，例如 `pts/0`
TPGID	当前占用终端的前台进程组
STAT `S`	可中断睡眠
STAT `+`	前台进程组
STAT `T`	暂停
STAT `s`	会话领导者

没有控制终端时，TTY 通常显示为 `?`。

2.2 登录会话与终端

2.2.1 SSH 登录链路

```
Xshell
  -> sshd
  -> 身份认证
  -> 分配 /dev/pts/N
  -> 启动 bash
  -> 建立带 SID 的登录会话
```

```
ZL@VM-0-14-ubuntu:~$ ps ajx | grep sshd
      1     933     933     933 ?             -1 Ss       0   0:56 sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 star
ups
    933  186429  186429  186429 ?             -1 Ss       0   0:00 sshd: ZL [priv]
 186429  186511  186429  186429 ?             -1 S     1002   0:00 sshd: ZL@pts/15
    933  204853  204853  204853 ?             -1 Ss       0   0:00 sshd: ZL [priv]
    933  204854  204854  204854 ?             -1 Ss       0   0:00 sshd: ZL [priv]
 204854  205074  204854  204854 ?             -1 S     1002   0:00 sshd: ZL@pts/6
 204853  205164  204853  204853 ?             -1 S     1002   0:00 sshd: ZL@pts/16
 205075  205434  205433  205075 pts/6     205433 S+    1002   0:00 grep --color=auto sshd
```

`/etc/passwd` 保存用户名、用户 ID、家目录和登录 shell，密码摘要保存在 `/etc/shadow`。认证通过后，系统为用户启动 shell 并分配终端文件。

```
ZL@VM-0-14-ubuntu:~$ cat /etc/password
cat: /etc/password: No such file or directory
ZL@VM-0-14-ubuntu:~$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:998:998:systemd Network Management:/:/usr/sbin/nologin
systemd-timesync:x:997:997:systemd Time Synchronization:/:/usr/sbin/nologin
dhcpcd:x:100:65534:DHCP Client Daemon,,,:/usr/lib/dhcpcd:/bin/false
```

同一终端中的多个作业可以有不同 PGID，但通常属于同一个 SID。终端文件既是输入源，也是输出目标。

```
##############
   PPID     PID    PGID     SID TTY        TPGID STAT   UID   TIME COMMAND
 376142  376145  376145  376145 pts/6     379400 Ss    1002   0:00 -bash
 377628  377629  377629  377629 pts/15    377629 Ss+   1002   0:00 -bash
      1 1835700 1835700 1835700 pts/11   1835700 Ss+   1002   0:00 /bin/bash --init-file /h
25271b67ce1df75/server/out/vs/workbench/contrib/terminal/common/scripts/shellIntegration-ba
      1 2714641 2714641 2714641 pts/9    2714641 Ss+   1002   0:00 /bin/bash --init-file /h
25271b67ce1df75/server/out/vs/workbench/contrib/terminal/common/scripts/shellIntegration-ba
      1 2720079 2720079 2720079 pts/10   2720079 Ss+   1002   0:00 /bin/bash --init-file /h
25271b67ce1df75/server/out/vs/workbench/contrib/terminal/common/scripts/shellIntegration-ba
total 0
drwxr-xr-x  2 root root       0 Jul  3 23:15 ./
drwxr-xr-x 19 root root    3960 Jul  3 23:15 ../
crw--w----  1 root tty  136,  1 Jul  4 10:11 1
crw--w----  1 ZL   tty  136, 10 Aug  7 10:16 10
crw--w----  1 ZL   tty  136, 11 Aug  3 21:46 11
crw--w----  1 ZL   tty  136, 15 Sep 18 08:56 15
crw--w----  1 root tty  136,  4 Jul  4 10:16 4
crw--w----  1 ZL   tty  136,  6 Sep 18 08:56 6
crw--w----  1 ZL   tty  136,  8 Aug  7 13:25 8
crw--w----  1 ZL   tty  136,  9 Aug  7 10:16 9
c---------  1 root root   5,  2 Jul  3 23:15 ptmx
```

2.2.2 注销为什么影响服务

关闭终端、注销会话或断开 SSH，可能使会话中的进程收到 SIGHUP。直接从前台或后台作业启动的 TCP 服务并不是真正的独立服务，用户退出可能导致服务终止。

后台运行只改变作业的前后台关系，不能自动脱离登录会话。

3. 守护进程与长期运行

3.1 setsid

`setsid()` 创建新会话，使调用进程成为会话领导者。调用进程不能已经是所在进程组组长，因此常先 `fork`，让父进程退出，再由子进程调用 `setsid`。

```
setsid(2)                                                  System Calls Manual

NAME
       setsid - creates a session and sets the process group ID

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <unistd.h>

       pid_t setsid(void);
```

```c
pid_t pid = fork();
if (pid < 0) {
    // 处理错误
```

```
} else if (pid > 0) {
    exit(0);
}

setsid();
```

调用后，进程与原来的控制终端脱离，不再因为原登录会话结束而直接停止。

glibc 还提供了封装接口：

```
#include <unistd.h>

int daemon(int nochdir, int noclose);
```

`daemon(0, 0)` 会把工作目录切换到 `/`，并把标准输入、输出和错误重定向到 `/dev/null`；非零的 `nochdir` 或 `noclose` 会保留对应行为不变。

3.2 标准流、工作目录与日志

守护进程通常把标准输入、输出和错误重定向到 `/dev/null`：

```
int fd = open("/dev/null", O_RDWR);
dup2(fd, STDIN_FILENO);
dup2(fd, STDOUT_FILENO);
dup2(fd, STDERR_FILENO);
```

```
ZL@VM-0-14-ubuntu:~$ ll /dev/null
crw-rw-rw- 1 root root 1, 3 Jul  3 23:15 /dev/null
ZL@VM-0-14-ubuntu:~$ ll /dev/zero
crw-rw-rw- 1 root root 1, 5 Jul  3 23:15 /dev/zero
ZL@VM-0-14-ubuntu:~$
```

工程中不能因此放弃日志。日志应写入文件或日志系统，支持轮转、级别和故障定位。

稳定工作目录也很重要：

```
chdir("/");
```

这样服务不会一直占用可卸载目录，也更容易通过绝对路径定位配置、日志和数据。

3.3 部署目录

一个可部署的 TCP 服务通常包含：

```
bin/        可执行文件
conf/       配置
log/        日志
wwwroot/    静态数据或业务资源
```

```
ZL@VM-0-14-ubuntu:~/code/lesson41/HTTP$ ps ajx | grep httpserver && ps ajx | head -1
      1  427224  427224  427224 ?             -1 Ss    1002   0:00 httpserver 8080
 421690  428424  428423  421690 pts/6     428423 S+    1002   0:00 grep --color=auto httpserver
   PPID     PID    PGID     SID TTY        TPGID STAT   UID   TIME COMMAND
```

```
lrwxrwxrwx  1 ZL ZL  0 Sep 18 12:10 cwd -> /home/ZL/code/lesson41/HTTP
-r--------  1 ZL ZL  0 Sep 18 12:10 environ
lrwxrwxrwx  1 ZL ZL  0 Sep 18 12:10 exe -> /usr/bin/httpserver
```

```
ZL@VM-0-14-ubuntu:~/code/lesson41/HTTP$ ls /proc/427224/fd -l
total 0
lrwx------ 1 ZL ZL 64 Sep 18 12:18 0 -> /dev/null
lrwx------ 1 ZL ZL 64 Sep 18 12:18 1 -> /dev/null
lrwx------ 1 ZL ZL 64 Sep 18 12:18 2 -> /dev/null
lrwx------ 1 ZL ZL 64 Sep 18 12:18 3 -> 'socket:[1390059925]'
```

```
daemon(3)

NAME
       daemon - run in the background

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <unistd.h>

       int daemon(int nochdir, int noclose);
```

通过 `ps ajx` 观察时，守护进程通常自成会话和进程组，TTY 为 `?`。用户注销不应直接结束这个独立会话。

3.4 服务重启与地址复用

服务端频繁重启时，旧连接可能仍占用本地端口，导致 `bind` 失败。可以设置：

```
int opt = 1;
setsockopt(
    listenfd,
    SOL_SOCKET,
    SO_REUSEADDR,
    &opt,
    sizeof(opt)
);
```

```
getsockopt(2)

NAME
       getsockopt, setsockopt - get and set options on sockets

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <sys/socket.h>

       int getsockopt(int sockfd, int level, int optname,
                      void optval[restrict *.optlen],
                      socklen_t *restrict optlen);
       int setsockopt(int sockfd, int level, int optname,
                      const void optval[.optlen],
                      socklen_t optlen);
```

TIME_WAIT 的原因、CLOSE_WAIT 和端口状态诊断见 2026-9-18 Linux网络--TCP连接管理与异常诊断。

4. Socket API

4.1 文件描述符与地址

4.1.1 socket

```
int sockfd = socket(AF_INET, SOCK_STREAM, 0);
```

- `AF_INET`：IPv4；
- `SOCK_STREAM`：TCP；
- 成功返回文件描述符，失败返回 -1。

Linux socket 遵循文件抽象，应用可用 `read/write` 或 `recv/send` 操作。`write` 成功后数据通常进入内核发送缓冲区，不表示对端应用已读取。

4.1.2 sockaddr_in

```
struct sockaddr_in addr;
memset(&addr, 0, sizeof(addr));
addr.sin_family = AF_INET;
addr.sin_port = htons(port);
addr.sin_addr.s_addr = htonl(INADDR_ANY);
```

`htons`、`htonl` 完成主机字节序到网络字节序转换。`INADDR_ANY` 表示监听本机所有可用地址。

`struct sockaddr *` 是通用地址指针，`sockaddr_in` 是 IPv4 具体结构，调用时还需传入结构体实际长度。

4.2 服务端

4.2.1 socket、bind、listen

```
int listenfd = socket(AF_INET, SOCK_STREAM, 0);
bind(listenfd, (struct sockaddr *)&local, sizeof(local));
listen(listenfd, backlog);
```

- `bind` 绑定本地 IP 和端口；
- `listen` 让内核能够接收连接请求；
- `backlog` 与等待应用接管的连接数量有关；
- `listenfd` 继续用于监听，不用于和客户端通信。

4.2.2 accept

```
int connfd = accept(listenfd, NULL, NULL);
```

`accept` 从全连接队列取出已完成三次握手的连接，返回新的 `connfd`。它不执行握手；如果没有已完成连接，默认阻塞。

队列实验、`SYN_RECV` 和 tcpdump 抓包见 2026-9-18 Linux网络--TCP连接管理与异常诊断。

4.3 客户端

4.3.1 connect

```
connect(sockfd, (struct sockaddr *)&server, sizeof(server));
```

客户端通常不需要显式 `bind`，内核在连接时自动选择临时源 IP 和端口。`connect` 请求内核发起握手，成功后与已连接 socket 关联。

4.3.2 数据收发

```
write/send -> 发送缓冲区 -> TCP -> 网络
read/recv  <- 接收缓冲区 <- TCP <- 网络
```

应用需要处理：

- 短写：`send/write` 只发送部分数据，应循环发送；
- 短读：一次读取可能不足一条消息；
- 一次读取多条消息；
- `read` 返回 0 表示对端有序关闭；
- 返回负值要结合 `errno` 判断。

应用消息边界方案见 2026-9-18 Linux网络--TCP协议基础与报文。

5. 服务端并发模型

5.1 单连接版本

```
listen -> accept -> read/write loop -> close
```

一个连接进入 `read` 循环后，主循环无法继续 `accept`，其他客户端只能等待。

5.2 多进程版本

每次 `accept` 后 `fork` 子进程处理连接：

- 子进程关闭不需要的 `listenfd`；
- 父进程关闭已经交给子进程的 `connfd`；
- 父进程继续 `accept`；
- 通过 `waitpid` 或信号处理回收子进程。

进程隔离强，但创建和回收成本高。

5.3 多线程版本

主线程持续 `accept`，每个连接创建线程处理。线程共享地址空间，需要正确传递 `connfd`、客户端地址和任务生命周期。

线程结束时必须关闭 `connfd` 并释放任务对象，否则会产生文件描述符和内存泄漏。

5.4 线程池版本

主线程把连接任务放入线程池，固定数量工作线程循环处理任务。它避免无限创建线程，是高并发服务常见结构之一。

线程池仍需解决：

- 任务队列上限与拒绝策略；
- 连接关闭；
- 异常任务不导致工作线程退出；
- 服务器停止时如何回收资源。

5.5 长连接处理

交互式长连接可以在一个线程中循环 `read/write`。为避免单线程无限阻塞，也可以使用非阻塞 socket、`select/poll/epoll` 或异步框架。

6. Windows TCP 客户端

6.1 Winsock 初始化

```
WSAStartup
  -> socket(AF_INET, SOCK_STREAM, IPPROTO_TCP)
  -> connect
  -> send/recv
  -> closesocket
  -> WSACleanup
```

如果使用 Winsock2：

```
#include <winsock2.h>
#pragma comment(lib, "ws2_32.lib")
```

6.2 Linux 与 Windows 差异

Linux/POSIX	Windows Winsock
socket 为 `int`	类型为 `SOCKET`
失败返回 -1	失败为 `INVALID_SOCKET`
无需初始化网络库	`WSAStartup`
`close`	`closesocket`
无额外清理	`WSACleanup`
通常链接 libc	链接 `ws2_32.lib`

Windows 示例必须检查 `WSAStartup`、socket 和 connect 的返回值，并处理部分发送。跨主机测试还要检查 Linux 服务监听地址、云安全组、Windows 防火墙和端口。

7. 断线检测与重连

7.1 连接状态机

```
NEW
  -> CONNECTING
  -> CONNECTED
  -> DISCONNECTED -> 重连
  -> CLOSED
```

状态	含义
NEW	尚未建立连接
CONNECTING	正在发起连接
CONNECTED	连接可用
DISCONNECTED	连接失效，应尝试重连
CLOSED	主动关闭或重试达到上限

7.2 重连流程

```
connect 失败 / 检测到断线
  -> 关闭旧 fd
  -> 等待退避时间
  -> 新建 socket
  -> 再次 connect
  -> 成功：CONNECTED
  -> 失败：继续重试或达到上限
```

重试间隔可以采用指数退避，并设置最大间隔、最大次数和随机抖动，避免大量客户端同时重连。

7.3 示例的改进

原示例中：

- `socket()` 创建失败直接 `exit(1)`，重连机制失效；
- 只有 `read()==0` 被设为断线，读写错误可能直接终止；
- 客户端阻塞在 `getline` 时无法发现服务端已经失联。

更可靠的实现需要统一连接错误状态、处理部分读写，并使用超时读、事件循环或心跳线程。

7.4 应用层心跳

心跳定期发送轻量请求并检查响应：

```
成功响应 -> 更新活跃时间
连续超时 -> 关闭旧连接
          -> 进入重连状态
```

TCP keepalive 和业务心跳的区别见 2026-9-18 Linux网络--TCP连接管理与异常诊断。

8. 内核中的 socket

8.1 文件描述符链路

```
进程 fd 表
  -> struct file
  -> private_data
  -> struct socket
  -> struct sock / inet_sock / tcp_sock
```

应用拿到的是文件描述符，内核通过文件对象和通用 socket 找到具体 TCP 连接对象。

8.2 tcp_sock 保存的状态

TCP 连接对象保存：

- 发送序号和接收序号；
- 接收窗口、拥塞窗口和 `ssthresh`；
- MSS 和协议选项；
- 重传、延迟应答和保活计时器；
- 最近活跃时间；
- 收发队列和等待进程。

8.3 sk_buff

报文在内核中由 `sk_buff` 描述。接收到的数据进入接收队列，发送数据进入发送队列；逻辑上可以看作连续字节流，物理上由队列和分片结构管理。

应用读取时如果没有数据，会进入 socket 等待队列；底层收到数据后通过回调唤醒进程。

9. 工程实践检查表

9.1 地址与连接

- 服务端是否使用合适的监听地址；
- `INADDR_ANY` 是否符合部署需求；
- 是否设置合理 backlog；
- 是否及时 `accept`；
- 是否处理连接失败和安全组/防火墙。

9.2 数据收发

- `send` 是否处理部分发送；
- `recv` 如何处理 0 和负值；
- 是否定义应用消息边界；
- 发送缓冲区大小是否合适；
- 大消息是否避免长期占用单个工作线程。

9.3 资源管理

- 每个 `accept` 返回的 fd 最终是否关闭；
- 多进程是否处理僵尸进程；
- 多线程是否正确传递和释放任务对象；
- 是否监控 CLOSE_WAIT 和文件描述符数量；
- 日志、配置和运行目录是否稳定。

9.4 断线与重连

- 是否启用心跳；
- 是否设置连接超时；
- 重连是否有上限和退避；
- 服务恢复后是否重置重试状态；
- 是否存在重连风暴。

10. 小结

TCP Socket 编程把协议状态映射为文件描述符操作：

```
socket
  -> bind/listen
  -> connect / accept
  -> read/write
  -> shutdown / close
```

协议层负责握手、ACK、重传、窗口和状态机，应用层负责消息边界、业务处理、异常返回和资源释放。Linux 服务还需要脱离登录会话成为守护进程，并正确处理日志、配置和路径。Windows 客户端、断线重连和心跳只是同一套 TCP 语义在跨平台与长期运行场景下的工程化延伸。